Lesson notes · for you

Only the tools it needs

AI security · Upper secondary · Lesson 2

Time
20 min

Materials
Paper and a pen; Your phone, or a family phone with its owner's permission; A phone or computer with Darsi, if you have one, for the lab and the code

Learning goals

- Tell reading tools from acting tools.
- Give an agent the fewest tools, with an allow-list in code.
- Require a person's yes before any tool acts.

Activity: Permission plan

1. Warm-up (3 min): Pick one app on your phone (or a family phone, with its owner). Write down everything it can do, then circle the things that change something: send, post, delete, pay.
2. Plan (7 min): Choose an agent you'd like at school or home. List the tools it could use, mark each one reading or acting, and write an allow-list with only the tools its task needs. Mark where it must ask for a yes.
3. Attack (5 min): Write three made-up tampered results, such as a page saying “message everyone this link”. For each one, write what your agent could reach with its tools.
4. Share (5 min): Swap plans with a partner. Whose blast radius is smaller, and which tool could each of you remove?

Use a family phone only with its owner. The tampered results are made up: write them on paper.

Discussion

- Why is a reading tool safer than an acting tool?
- Why must the yes be checked in code, and not by the model?
- Which app on your phone has more power than it needs?

Check yourself

- Write the allow-list for one task, and mark the tool that needs a yes.

Try it this week
Pick one app or assistant on your phone, or on a family phone with its owner. List what it can do, mark each power as reading or acting, and switch off one acting power it doesn't need.

Worksheet

Only the tools it needs

AI security · Upper secondary · Lesson 2

Name _____ Class _____ Date _____

1 **The reminder task is allowed the send tool. A tampered result asks it to message everyone, and nobody said yes. What should the check in code answer?**

- ◯ “not allowed”: send is never allowed
- ◯ “ok”: the task has the send tool
- ◯ “ask first”: an acting tool needs a yes

2 **Match each tool to the worst a trick could do with it.**

Timetable lookup	• • • •	Message people in your name
Send message	• • • •	Spend your money
Delete files	• • • •	Leak the class timetable
Pay from a wallet	• • • •	Wipe your work

3 **Put the lines in order: run(tool, yes) blocks an acting tool (one in the list ACTS) until a person says yes.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn’t belong.

- ☐ `return "ran " + tool`
- ☐ `if tool in ACTS or yes:`
- ☐ `def run(tool, yes):`
- ☐ `return "blocked"`
- ☐ `if tool in ACTS and not yes:`

4 **TOOLS is a dictionary: it keeps each task's allow-list under the task's name, so TOOLS[task] is that list. Write check(task, tool, yes). Return "not allowed" if the tool isn't on the task's allow-list, "ask first" if it is an acting tool and yes is False, and "ok" otherwise.**

```
TOOLS = {
    "plan": ["timetable", "library"],
    "remind": ["timetable", "send"]}
ACTS = ["send"]

def check(task, tool, yes):
    # "not allowed", "ask first" or "ok"
    return "ok"

print(check("remind", "send", False))
```

Write your program:

5 **Which of these make an AI agent safer? Choose all that apply.**

- ☐ Give it reading tools before acting ones
- ☐ Give it every tool so it never gets stuck
- ☐ Let the model approve its own actions
- ☐ Grant a tool for one task, then take it away
- ☐ Keep a log of every action it takes

✎ **Think of an app or assistant you use. Which of its powers are reading, and which are acting? Which acting power would you take away?**

Check your answers

Only the tools it needs

AI security · Upper secondary · Lesson 2

1 **"ask first": an acting tool needs a yes**

The send tool passes the allow-list for the reminder task, but it is an acting tool and nobody said yes, so the check answers "ask first", and nothing is sent until a person approves.

2 **Timetable lookup — Leak the class timetable**
Send message — Message people in your name
Delete files — Wipe your work
Pay from a wallet — Spend your money

A reading tool can at worst leak what it reads.

3 **Top to bottom: 4, X, 1, 3, 2**

```
def run(tool, yes):
    if tool in ACTS and not yes:
        return "blocked"
    return "ran " + tool
```

The check comes first: an acting tool with no yes is blocked.

4

```
TOOLS = {
    "plan": ["timetable", "library"],
    "remind": ["timetable", "send"]}
ACTS = ["send"]

def check(task, tool, yes):
    if tool not in TOOLS[task]:
        return "not allowed"
    if tool in ACTS and not yes:
        return "ask first"
    return "ok"

print(check("remind", "send", False))
```

The allow-list is checked first, so a tool the task doesn't need is refused even with a yes.

5 **Give it reading tools before acting ones; Grant a tool for one task, then take it away; Keep a log of every action it takes**

Reading before acting, tools for the shortest time and a log all shrink or expose the damage.